

Rekursion in Prolog: Beliebige Stammbaumtiefen beherrschen

Das 2-Säulen-Prinzip (Basisfall & Rekursionsschritt) – Generalprobe für die Klassenarbeit

Name:

Datum:

Klasse: Diff-10

THEMENSCHWERPUNKT: REKURSIVE INFERENZ & TRACING

MERKE

1. Warum scheitern starre Regeln? Das 2-Säulen-Prinzip der Rekursion

Mit starren Regeln wie `grossvater/2` können wir nur genau 2 Generationen überbrücken. Für Urgroßväter, Ur-Urgroßväter etc. bräuchten wir unendlich viele neue Regeln. **Die Lösung:** Eine *rekursive Definition*, die sich selbst aufruft, bis der Stammbaum endet!

Die 2 Säulen des Prädikats `vorfahre(X, Y) :`

Säule 1: Der Basisfall (Abbruch)

„Jeder direkte Elternteil ist sofort ein Vorfahre.“

`vorfahre(X, Y) :- elternteil(X, Y).`

Säule 2: Der Rekursionsschritt (Kette)

„X ist Vorfahre von Y, wenn X Elternteil von Z ist UND Z Vorfahre von Y ist.“

`vorfahre(X, Y) :- elternteil(X, Z), vorfahre(Z, Y).`

Ketten-Modell: $X \xrightarrow{\text{elternteil}} Z \xrightarrow{\text{vorfahre (Restkette)}} Y$. In jedem Schritt wird das Problem um eine Generation kleiner!

1 Verständnisprüfung: Die zwei Säulen der Rekursion

4 Punkte

Beantworte die Fragen zum fundamentalen Aufbau rekursiver Prädikate:

a) Welche Rolle spielt der **Basisfall** bei einer Rekursion? Was würde passieren, wenn man ihn weglässt?

Der Basisfall dient als
Ohne ihn könnte die Rekursion niemals

b) Welche Aufgabe hat die Variable **Z** im **Rekursionsschritt**?

Variable Z fungiert als

2 Klausur-Kernkompetenz: Schrittweises Tracing-Protokoll

6 Punkte

Gegeben ist der Götter-Stammbaum über 4 Generationen (Auszug):

`elternteil(uranos, kronos). elternteil(kronos, zeus). elternteil(zeus, ares). elternteil(ares, harmonia).`

Ein Benutzer stellt die Anfrage: `?- vorfahre(uranos, ares).`

Verfolge, wie Prolog die beiden Regeln Schritt für Schritt abarbeitet. Ergänze die Lücken im Protokoll:

Ebene / Aufruf	Was prüft Prolog? (Regel & Teilziel)	Ergebnis & Variablenbindung
Ebene 1: <code>vorfahre(uranos, ares)</code>	1. Säule 1 (Basisfall): Prüfe <code>elternteil(uranos, ares)</code> . 2. Säule 2 (Rekursion): Suche Kind <code>Z</code> : <code>elternteil(uranos, Z)</code> .	1. Scheitert (.....) 2. Treffer: <code>Z</code> = → Neuer Aufruf:
Ebene 2:	1. Säule 1 (Basisfall): Prüfe <code>elternteil(kronos, ares)</code> . 2. Säule 2 (Rekursion): Suche Kind <code>Z</code> : <code>elternteil(kronos, Z)</code> .	1. Scheitert (.....) 2. Treffer: <code>Z</code> = → Neuer Aufruf:
Ebene 3:	1. Säule 1 (Basisfall): Prüfe <code>elternteil(zeus, ares)</code> . (Greift der Basisfall bereits?)	Erfolg! Treffer als Fakt in Wissensbasis! Säule 1 liefert:
Rücklauf: Synthese	Die Erfolgsmeldung wandert die Aufrufkette zurück (Ebene 3 → Ebene 2 → Ebene 1).	Gesamtergebnis von Prolog:

Klausur-Fallen & Rekursions-Transfer

MERKE

2. Die zwei tödlichen Klausurfällen der Rekursion (Fehlervermeidung)

In Klassenarbeiten führen vor allem zwei typische Denkfehler zum sofortigen Programmabsturz:

Falle A: Die Linksrekursion (Left Recursion)

```
vorfahre(X, Y) :- vorfahre(X, Z), elternteil(Z, Y).
```

Problem: Prolog ruft `vorfahre` erneut auf, *bevor* `Z` gebunden ist! → **Endlosschleife / Stack Overflow!**

Falle B: Vergessener Basisfall

Wer nur den Rekursionsschritt notiert, verhindert das Erkennen direkter Elternbeziehungen und beraubt Prolog der Abbruchmarke.
→ Prolog kann niemals erfolgreich stoppen!

• Die eiserne Klausur-Regel: Im Rekursionsschritt steht immer zuerst der Fakten-Schritt (`elternteil(X, Z)`), der die Variable bindet, und erst danach der rekursive Aufruf (`vorfahre(Z, Y)`)!

3 Fehlerdetektive: Klausur-Code analysieren & korrigieren

5 Punkte

Zwei Schüler haben in der Vorbereitung auf die Arbeit folgenden Code geschrieben. Finde und erkläre die Fehler:

a) Fall Tim (Absturz bei Anfrage):

Tim hat folgende Regel deklariert: `vorfahre(X, Y) :- vorfahre(X, Z), elternteil(Z, Y).`

Erkläre, warum Prolog bei der Anfrage `?- vorfahre(kronos, ares).` abstürzt und wie die Zeile korrekt lauten muss.

Erklärung: Es liegt eine unzulässige _____ vor. Prolog ruft sich selbst auf, ohne dass Variable Z _____ . Dadurch entsteht eine _____ .

Korrektur: `vorfahre(X, Y) :- _____`

b) Fall Sara (Der fehlende Basisfall):

Sara hat den Basisfall vergessen und nur den Rekursionsschritt in ihrer Datei stehen.

Was antwortet Prolog auf die direkte Anfrage `?- vorfahre(zeus, ares).` (Zeus ist direkter Vater von Ares)? Begründe.

Antwort von Prolog: _____

Begründung: _____

4 Klausur-Generalprobe: Modellierung eines Schienennetzes

5 Punkte

Gegeben sind Fakten über Zugverbindungen zwischen Bahnhöfen: `schiene(berlin, hannover).`

`schiene(hannover, dortmund).` `schiene(dortmund, koeln).`

Formuliere das rekursive Prädikat `erreichbar(Von, Nach)`, das prüft, ob man von Bahnhof *Von* nach *Nach* reisen kann (über beliebig viele Zwischenhalte):

a) Säule 1 (Basisfall: Direkte Schienenverbindung):

`erreichbar(Von, Nach) :- _____`

b) Säule 2 (Rekursionsschritt: Fahrt über Zwischenstation Z):

`erreichbar(Von, Nach) :- _____`

c) Beweisführung: Welche Zwischenstationen bindet Prolog bei der Anfrage `?- erreichbar(berlin, koeln).`?

Kette der Halte: berlin → _____ → _____ → koeln

Interaktiver Rekursions-Simulator auf dem iPad: Nutze im Schulportal das neue Modul „Stunde 4 • Rekursion & Stammbäume“, um den Variablen-Stack und den Rücklauf Schritt für Schritt live zu animieren!

WEB/PROLOG-
REKURSION.HTML